

```
// An optimized version of Bubble Sort
void bubbleSort(int arr[], int n){
    int i, j;
    bool swapped;
    for (i = 0; i < n - 1; i++) {
        swapped = false;
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(&arr[j], &arr[j + 1]);
                swapped = true;
            }
        }
        //printf("array after iteration i = %d: \n",i);    //printArray(arr, n);

        // If no two elements were swapped by inner loop, then break
        if (swapped == false)
        {
            printf("No more comparisons needed - breaking \n");
            break;
        }
    }
}
```

```
/* Function to sort array using insertion sort */
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        /* Move elements of arr[0..i-1], that are greater than key, to one position ahead of their
        current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;

        printf("array after iteration i = %d: \n",i);
        printArray(arr, n);
    }
}
```

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {

        // Assume the current position holds the minimum element
        int min_idx = i;

        // Iterate through the unsorted portion to find the actual minimum
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {

                // Update min_idx if a smaller element is found
                min_idx = j;
            }
        }

        // Move minimum element to its correct position
        int temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;

        printf("array after iteration i = %d: \n",i);
        printArray(arr, n);
    }
}
```

QUICKSORT( $A, p, r$ )

```
1  if  $p < r$ 
2      then  $q \leftarrow \text{PARTITION}(A, p, r)$ 
3          QUICKSORT( $A, p, q - 1$ )
4          QUICKSORT( $A, q + 1, r$ )
```

To sort an entire array  $A$ , the initial call is QUICKSORT( $A, 1, \text{length}[A]$ ).

### Partitioning the array

The key to the algorithm is the PARTITION procedure, which rearranges the subarray  $A[p \dots r]$  in place.

PARTITION( $A, p, r$ )

```
1   $x \leftarrow A[r]$ 
2   $i \leftarrow p - 1$ 
3  for  $j \leftarrow p$  to  $r - 1$ 
4      do if  $A[j] \leq x$ 
5          then  $i \leftarrow i + 1$ 
6              exchange  $A[i] \leftrightarrow A[j]$ 
7  exchange  $A[i + 1] \leftrightarrow A[r]$ 
8  return  $i + 1$ 
```

```
// The QuickSort function implementation
void quickSort(int arr[], int low, int high) {
    printf("In Quicksort low = %d and high = %d ",low,high);
    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        // recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```
// partition function
int partition(int arr[], int low, int high) {
    // Choose the pivot
    int pivot = arr[high];

    // Index of smaller element and indicates the right position of pivot found so far
    int i = low - 1;

    // Traverse arr[low..high] and move all smaller elements to the left side.
    // Elements from low to i are smaller after every iteration
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    // Move pivot after smaller elements and
    // return its position
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
```

```
void build_maxheap(int Arr[ ], int size)
{
    //printArray(Arr, size);
    int N=size;
    for(int i = (N/2-1) ; i >= 0 ; i-- )
    {
        //max_heapify (Arr, i) ;
        heapify(Arr, size, i);
    }
}
```

```
void heap_sort(int Arr[], int size)
{
    int heap_size = size;
    build_maxheap(Arr, size);
    for(int i = size-1; i>=1 ; i-- )
    {
        swap(&Arr[ 0 ], &Arr[ i ]);
        heap_size = heap_size-1;
        //max_heapify(Arr, 1, heap_size);
        heapify(Arr, heap_size, 0);
    }
}
```

```
void heapify(int array[], int size, int i) {  
    if (size == 1) {  
        //printf("Single element in the heap");  
        return;  
    } else {  
        // Find the largest among root, left child and right child  
        int largest = i;  
        int l = 2 * i + 1;  
        int r = 2 * i + 2;  
        if (l < size && array[l] > array[largest])  
            largest = l;  
        if (r < size && array[r] > array[largest])  
            largest = r;  
  
        // Swap and continue heapifying if root is not largest  
        if (largest != i) {  
            swap(&array[i], &array[largest]);  
            heapify(array, size, largest);  
        }  
    }  
}
```

```
// l is for left index and r is right index of the
// sub-array of arr to be sorted
void mergeSort(int arr[], int l, int r){
    //printf("Calling mergeSort with l=%d r=%d \n",l,r);
    if (l < r) {
        int m = l + (r - l) / 2;

        // Sort first and second halves
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);

        merge(arr, l, m, r);
    }
}
```

MERGE-SORT( $A, p, r$ )

```
1  if  $p < r$ 
2      then  $q \leftarrow \lfloor (p + r)/2 \rfloor$ 
3          MERGE-SORT( $A, p, q$ )
4          MERGE-SORT( $A, q + 1, r$ )
5          MERGE( $A, p, q, r$ )
```

MERGE( $A, p, q, r$ )

```
1   $n_1 \leftarrow q - p + 1$ 
2   $n_2 \leftarrow r - q$ 
3  create arrays  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$ 
4  for  $i \leftarrow 1$  to  $n_1$ 
5      do  $L[i] \leftarrow A[p + i - 1]$ 
6  for  $j \leftarrow 1$  to  $n_2$ 
7      do  $R[j] \leftarrow A[q + j]$ 
8   $L[n_1 + 1] \leftarrow \infty$ 
9   $R[n_2 + 1] \leftarrow \infty$ 
10  $i \leftarrow 1$ 
11  $j \leftarrow 1$ 
12 for  $k \leftarrow p$  to  $r$ 
13     do if  $L[i] \leq R[j]$ 
14         then  $A[k] \leftarrow L[i]$ 
15              $i \leftarrow i + 1$ 
16         else  $A[k] \leftarrow R[j]$ 
17              $j \leftarrow j + 1$ 
```

```
// Merges two subarrays of arr[].
// First subarray is arr[l..m]
// Second subarray is arr[m+1..r]
void merge(int arr[], int l, int m, int r){

    printf("Calling merge with l=%d  m=%d  r=%d \n",l,m,r);

    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;

    // Create temp arrays
    int L[n1], R[n2];

    // Copy data to temp arrays L[] and R[]
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
```

```
// Merge the temp arrays back into arr[l..r
i = 0;  j = 0;  k = l;
while (i < n1 && j < n2) {
    if (L[i] <= R[j]) {
        arr[k] = L[i];
        i++;
    }
    else {
        arr[k] = R[j];    j++;
    }
    k++;
}
// Copy the remaining elements of L[]  // if there are any
while (i < n1) {
    arr[k] = L[i];    i++;
    k++;
}
// Copy the remaining elements of R[]  // if there are any
while (j < n2) {
    arr[k] = R[j];
    j++;
    k++;
}
}
```

| Algorithm      | Best Time     | Average Time  | Worst Time    | Space Complexity | Stable? |
|----------------|---------------|---------------|---------------|------------------|---------|
| Bubble Sort    | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$           | Yes     |
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$           | Yes     |
| Selection Sort | $O(n^2)$      | $O(n^2)$      | $O(n^2)$      | $O(1)$           | No      |
| Heap Sort      | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$           | No      |
| Merge Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(n)$           | Yes     |
| Quick Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$      | No      |

```
int linearSearch(int arr[], int n, int target) {  
  
    // Iterate linearly through the array  
    for (int i = 0; i < n; i++)  
        if (arr[i] == target)  
            return i;  
    return -1;  
}
```

```
int binarySearch(int arr[], int n, int x) {  
    int low = 0;  
    int high = n-1;  
    while (low <= high) {  
        int mid = low + (high - low) / 2;  
  
        // Check if x is present at mid  
        if (arr[mid] == x)  
            return mid;  
  
        // If x greater, ignore left half  
        if (arr[mid] < x)  
            low = mid + 1;  
  
        // If x is smaller, ignore right half  
        else  
            high = mid - 1;  
    }  
  
    return -1;  
}
```

```
// A recursive binary search function. It returns location of x in given array
// arr[low..high] is present, otherwise -1
int binarySearchR(int arr[], int low, int high, int x) {
    if (high >= low) {
        int mid = low + (high - low) / 2;

        // If the element is present at the middle
        // itself
        if (arr[mid] == x)
            return mid;

        // If element is smaller than mid, then
        // it can only be present in left subarray
        if (arr[mid] > x)
            return binarySearchR(arr, low, mid - 1, x);

        // Else the element can only be present
        // in right subarray
        return binarySearchR(arr, mid + 1, high, x);
    }
    // We reach here when element is not
    // present in array
    return -1;
}
```

```

// Returns index of x if present, else returns -1
int fibonnaciSearch(int arr[], int n, int x) {

    // initialize first three fibonacci numbers
    int a = 0, b = 1, c = 1;

    // iterate while c is smaller than n.. c stores the smallest Fibonacci
    // number greater than or equal to n
    while (c < n) {
        a = b;    b = c;    c = a + b;
    }
    // marks the eliminated range from front
    int offset = -1;

    // while there are elements to be inspected Note that we compare arr[a] with x.
    // When c becomes 1, a becomes 0
    while (c > 1) {
        // check if a is a valid location
        int i = min(offset + a, n - 1);

        // if x is greater than the value at index a, cut the sub array from offset to i
        if (arr[i] < x) {
            c = b;    b = a;    a = c - b;    offset = i;
        }
    }
}

```

```
// else if x is greater than the value at
// index a, cut the subarray after i+1
else if (arr[i] > x) {
    c = a;
    b = b - a;
    a = c - b;
}

// else if element found, return index
else
    return i;
}

// comparing the last element with x
if (b && arr[offset + 1] == x)
    return offset + 1;

// element not found, return -1
return -1;
}
```

| Feature                 | Linear Search                | Iterative Binary Search                                                 | Recursive Binary Search                                                 | Fibonacci Search                                             |
|-------------------------|------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------|
| Data Requirement        | Unsorted or Sorted           | Must be <b>Sorted</b>                                                   | Must be <b>Sorted</b>                                                   | Must be <b>Sorted</b>                                        |
| Approach                | Sequential                   | Divide and Conquer                                                      | Divide and Conquer                                                      | Divide and Conquer (Golden Ratio split)                      |
| Best Time Complexity    | $O(1)$                       | $O(1)$                                                                  | $O(1)$                                                                  | $O(1)$                                                       |
| Average Time Complexity | $O(n)$                       | $O(\log n)$                                                             | $O(\log n)$                                                             | $O(\log n)$                                                  |
| Worst Time Complexity   | $O(n)$                       | $O(\log n)$                                                             | $O(\log n)$                                                             | $O(\log n)$                                                  |
| Space Complexity        | $O(1)$                       | $O(1)$                                                                  | <b><math>O(\log n)</math></b> (Call stack overhead)                     | $O(1)$                                                       |
| Primary Operators       | Equality ( <code>==</code> ) | Comparison ( <code>&lt;</code> , <code>&gt;</code> , <code>/ 2</code> ) | Comparison ( <code>&lt;</code> , <code>&gt;</code> , <code>/ 2</code> ) | Addition and Subtraction ( <code>+</code> , <code>-</code> ) |
| Hardware Fit            | Universal                    | Universal                                                               | Risk of Stack Overflow                                                  | Excellent Cache Locality                                     |